

AGROTANI MVP v2 — 30 Hari Realisasi

Iterasi ke-2. Arsitektur diupgrade: Firebase → **NestJS backend sendiri.**
Tetap realistis: **1 mahasiswa, 30 hari, budget minim.**

Apa yang Kita Bangun?

Aplikasi mobile yang bisa:

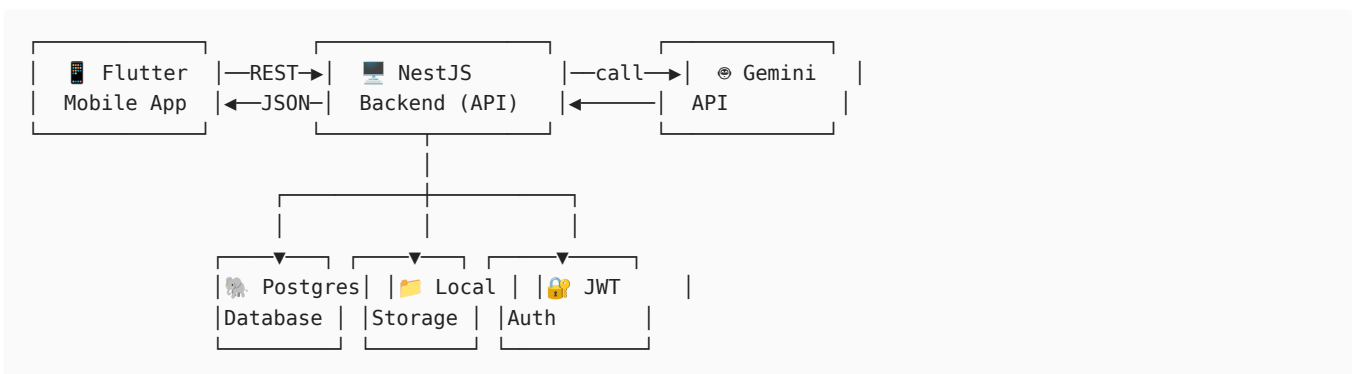
1.  **Scan tanaman** → foto daun/batang yang sakit
2.  **AI Diagnosis** → Gemini analisis foto, kasih tau penyakitnya apa
3.  **AI Rekomendasi** → kasih solusi (pupuk/pestisida/tindakan)
4.  **FarmerBot** → chatbot tanya jawab soal pertanian

Empat fitur. Tidak lebih.

Yang TIDAK Kita Bangun (di 30 hari ini)

- ~~Microservices / Kubernetes~~ → monolith NestJS saja
- ~~Smart Urgency System~~ → terlalu complex
- ~~Community Forum~~ → belum perlu
- ~~IoT / Sensor~~ → jauh
- ~~Weather Integration~~ → nanti
- ~~Marketplace~~ → nanti
- ~~Multi-language~~ → Bahasa Indonesia aja
- ~~Voice input~~ → nanti
- ~~Offline mode~~ → nanti

Arsitektur (v2 — NestJS Backend)



Kenapa pindah ke NestJS?

- Kontrol penuh atas backend (routing, logic, database)
- Belajar arsitektur backend yang proper (berguna di dunia kerja)
- Lebih gampang scale nanti dibanding Cloud Functions
- TypeScript end-to-end (NestJS backend = konsisten)
- PostgreSQL → relational, query lebih powerful dari Firestore

Kenapa masih tetap simpel?

- Tetap monolith (1 app NestJS, bukan microservices)
- Pakai SQLite/PostgreSQL lokal dulu, tidak perlu cloud DB

- Upload foto simpan di folder lokal server, bukan cloud storage
- Deploy di VPS murah (Rp 50-100rb/bulan) atau laptop sendiri saat demo

Tech Stack

Komponen	Teknologi	Kenapa
Frontend	Flutter	1 codebase → Android + iOS
Backend	NestJS (TypeScript)	Modular, decorators, mirip Angular → gampang dipelajari
Database	PostgreSQL	Relational, powerful query, gratis
ORM	Prisma	Type-safe, auto-generate, migrasi gampang
Auth	JWT (passport-jwt)	Simpel, stateless, standard industri
File Upload	Multer (lokal)	Simpan foto di server dulu, nanti pindah cloud
AI	Gemini 2.0 Flash	Murah, cepat, vision capable
HTTP Client (Flutter)	Dio	Interceptor, retry, gampang handle token

Timeline 30 Hari

Minggu 1 (Hari 1-7): Setup & Foundation

```
Hari 1-2: Setup Project
├─ Flutter project init + struktur folder
├─ NestJS project init (nest new agrotani-api)
├─ PostgreSQL setup (lokal / Docker)
├─ Prisma init + schema dasar
├─ Git repo (monorepo: /app + /api)
└─ Environment config (.env)

Hari 3-4: Auth System
├─ [Backend] POST /auth/register (email + password → hash bcrypt)
├─ [Backend] POST /auth/login → return JWT access token
├─ [Backend] GET /auth/profile → return user data (guarded)
├─ [Backend] Prisma: tabel users
├─ [Flutter] Login screen + Register screen
├─ [Flutter] Simpan token di SharedPreferences / flutter_secure_storage
└─ [Flutter] Dio interceptor: auto-attach token di setiap request

Hari 5-7: UI Dasar + Navigation
├─ Bottom navigation (Home, Scan, Chat, Profil)
├─ Home screen (placeholder dashboard)
├─ Scan screen (kamera + gallery picker)
├─ Chat screen (UI chat bubble)
├─ Profil screen (info user dari GET /auth/profile)
└─ Design system (warna, font, komponen reusable)
```

Minggu 2 (Hari 8-14): Fitur Scan & Diagnosis

```
Hari 8-9: Backend Scan API + Gemini
├─ [Backend] POST /scan/analyze
│   └─ Terima file gambar (Multer)
│   └─ Simpan gambar ke folder uploads/
│   └─ Convert gambar ke base64
│   └─ Kirim ke Gemini Vision API + prompt diagnosis
│   └─ Parse response Gemini
│   └─ Simpan hasil ke database (tabel scans)
├─ [Backend] GET /scan/history → list scan user
├─ [Backend] GET /scan/:id → detail satu scan
└─ [Backend] Prisma: tabel scans
```

Hari 10-12: Flutter Scan Flow

- └─ Ambil foto (camera / image_picker)
- └─ Preview foto → konfirmasi → kirim ke POST /scan/analyze
- └─ Loading screen dengan progress indicator
- └─ Tampilkan hasil diagnosis dari response
- └─ Simpan & navigasi ke detail screen

Hari 13-14: Hasil Diagnosis UI

- └─ Card diagnosis (nama penyakit, keparahan, confidence)
- └─ Card rekomendasi (langkah 1, 2, 3)
- └─ Riwayat scan (list dari GET /scan/history)
- └─ Detail scan (tap → full detail + foto asli)

Minggu 3 (Hari 15-21): FarmerBot & Polish

Hari 15-17: FarmerBot Chat

- └─ [Backend] POST /chat/send
 - └─ Terima message + chatId
 - └─ Ambil history chat dari DB
 - └─ Kirim ke Gemini Chat API (with history)
 - └─ Simpan message + response ke DB
 - └─ Return response
- └─ [Backend] GET /chat/sessions → list sesi chat
- └─ [Backend] GET /chat/:id/messages → history pesan
- └─ [Backend] Prisma: tabel chat_sessions + chat_messages
- └─ [Flutter] Chat UI dengan bubble user/bot
- └─ [Flutter] Quick reply buttons (3-4 pertanyaan umum)
- └─ [Flutter] Kirim foto di chat → diagnosis inline

Hari 18-19: Home Dashboard

- └─ Widget scan terakhir (dari GET /scan/history?limit=3)
- └─ Tombol quick scan (langsung buka kamera)
- └─ Tips harian (hardcoded 10-20 tips, tampil random)
- └─ Statistik sederhana (total scan dari profil)

Hari 20-21: UI Polish

- └─ Loading states yang baik di semua screen
- └─ Empty states (belum ada scan, chat kosong, dll)
- └─ Error handling UI (no internet, 500 error, timeout)
- └─ Animasi transisi dasar
- └─ Konsisten: icon, warna, spacing

Minggu 4 (Hari 22-30): Testing & Finalisasi

Hari 22-24: Testing

- └─ Test di HP Android low-end
- └─ Test semua flow: register → login → scan → chat → riwayat
- └─ Fix bugs
- └─ Test dengan foto tanaman nyata (sawah/kebun kampus)
- └─ Minta 3-5 teman test + kumpulkan feedback

Hari 25-27: Improvement dari Feedback

- └─ Fix UX issues dari testing
- └─ Improve prompt Gemini (akurasi diagnosis)
- └─ Optimasi: compress gambar sebelum upload
- └─ Handle edge cases (foto buram, bukan tanaman, timeout)

Hari 28-29: Dokumentasi & Persiapan Demo

- └─ Tulis README project (cara install, cara run)
- └─ Screenshot / screen recording
- └─ Siapkan slide presentasi
- └─ Build APK release
- └─ Deploy backend ke VPS / Railway / Render (gratis tier)

Hari 30: 🚀 DEMO DAY

- └ Demo ke dosen / teman
- └ Record video demo
- └ Upload ke GitHub

Prompt Engineering (Inti AI-nya)

System Prompt (Base)

Kamu adalah AgroAI, asisten pertanian Indonesia.
Jawab dalam Bahasa Indonesia sederhana yang mudah dipahami petani.
Gunakan emoji. Jangan pakai istilah terlalu teknis.
Jika tidak yakin, bilang "Saya kurang yakin" dan sarankan konsultasi ahli.
Jangan mengarang nama penyakit atau dosis obat yang tidak ada.

Prompt Scan/Diagnosis

Analisis foto tanaman ini. Berikan jawaban dalam format:

****🔍 Diagnosis:****

Nama penyakit atau masalah yang terdeteksi.

****📊 Tingkat Keparahan:****

Ringan / Sedang / Parah (pilih satu, jelaskan singkat)

****🍌 Rekomendasi:****

- Langkah 1: ...
- Langkah 2: ...
- Langkah 3: ...

****⚠ Catatan:****

Hal penting yang perlu diperhatikan.

Jika foto tidak jelas atau bukan tanaman, bilang bahwa foto perlu diambil ulang.
Konteks: Tanaman di Indonesia, iklim tropis.

Prompt FarmerBot

Kamu adalah FarmerBot, teman ngobrol petani yang ramah.
Jawab pertanyaan tentang pertanian dengan bahasa sederhana.
Berikan jawaban yang praktis dan bisa langsung dilakukan.
Jika user kirim foto, analisis dan kasih diagnosis singkat.
Akhiri jawaban dengan 1-2 saran pertanyaan lanjutan.

Struktur Folder

Flutter (Frontend)

```
app/
├─ lib/
│   ├── main.dart
│   ├── app.dart                # MaterialApp, theme, routing
│   └─ config/
│       ├── theme.dart          # Warna, font, style
│       ├── api_config.dart      # Base URL, timeout
│       └─ constants.dart
│   └─ models/
│       ├── user_model.dart
│       └─ scan_result_model.dart
```

```

└─ chat_message_model.dart

└─ services/
  └─ api_service.dart          # Dio instance + interceptor JWT
  └─ auth_service.dart         # Login, register, get profile
  └─ scan_service.dart         # Upload foto, get history
  └─ chat_service.dart         # Send message, get sessions

└─ screens/
  └─ auth/
    └─ login_screen.dart
    └─ register_screen.dart
  └─ home_screen.dart
  └─ scan/
    └─ scan_screen.dart
    └─ scan_loading_screen.dart
    └─ scan_result_screen.dart
  └─ chat/
    └─ chat_screen.dart
  └─ history/
    └─ history_screen.dart
  └─ profile/
    └─ profile_screen.dart

└─ widgets/
  └─ diagnosis_card.dart
  └─ recommendation_card.dart
  └─ chat_bubble.dart
  └─ quick_reply_button.dart
  └─ scan_history_tile.dart

└─ pubspec.yaml

```

NestJS (Backend)

```

api/
└─ src/
  └─ main.ts                  # Bootstrap, port, CORS
  └─ app.module.ts           # Root module

  └─ auth/
    └─ auth.module.ts
    └─ auth.controller.ts    # POST /register, /login, GET /profile
    └─ auth.service.ts       # Bcrypt hash, JWT sign/verify
    └─ jwt.strategy.ts        # Passport JWT strategy
    └─ dto/
      └─ register.dto.ts
      └─ login.dto.ts

  └─ scan/
    └─ scan.module.ts
    └─ scan.controller.ts    # POST /analyze, GET /history, GET /:id
    └─ scan.service.ts       # Gemini call, save result
    └─ dto/
      └─ analyze.dto.ts

  └─ chat/
    └─ chat.module.ts
    └─ chat.controller.ts    # POST /send, GET /sessions, GET /:id
    └─ chat.service.ts       # Gemini chat, manage history
    └─ dto/
      └─ send-message.dto.ts

  └─ gemini/
    └─ gemini.module.ts
    └─ gemini.service.ts     # Shared Gemini client (vision + chat)

```

```

├── common/
│   ├── guards/
│   │   └── jwt-auth.guard.ts
│   └── decorators/
│       └── current-user.decorator.ts
├── prisma/
│   ├── schema.prisma          # Database schema
│   └── migrations/
├── uploads/                   # Foto yang diupload (gitignore)
├── .env                       # DB_URL, GEMINI_API_KEY, JWT_SECRET
├── package.json
└── tsconfig.json

```

🗄 Database (PostgreSQL + Prisma)

```

// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String   @id @default(uuid())
  name        String
  email       String   @unique
  password    String   // hashed bcrypt
  createdAt   DateTime @default(now())

  scans       Scan[]
  chatSessions ChatSession[]
}

model Scan {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
  imageUrl    String   // path ke file di uploads/
  diagnosis   String   // nama penyakit
  severity    String   // Ringan / Sedang / Parah
  confidence  String   // misal "85%"
  recommendation String // langkah-langkah
  rawResponse String   // full response Gemini
  feedback    String?  // "accurate" / "inaccurate" / null
  createdAt   DateTime @default(now())
}

model ChatSession {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
  title       String   @default("Chat Baru")
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  messages ChatMessage[]
}

model ChatMessage {

```

```

id      String      @id @default(uuid())
sessionId String
session ChatSession @relation(fields: [sessionId], references: [id])
role    String      // "user" / "bot"
content String
imageUrl String?    // kalau user kirim foto
createdAt DateTime   @default(now())
}

```

🔧 API Endpoints

BASE URL: `http://localhost:3000/api`

```

— 🛡️ AUTH —
POST  /api/auth/register { name, email, password } → { accessToken }
POST  /api/auth/login   { email, password }       → { accessToken }
GET    /api/auth/profile [JWT required]         → { user data }

— 📷 SCAN —
POST  /api/scan/analyze [JWT] multipart: image file → { scan result }
GET    /api/scan/history [JWT] ?limit=10             → { scans[] }
GET    /api/scan/:id    [JWT]                       → { scan detail }
POST  /api/scan/:id/feedback [JWT] { feedback: "accurate" }

— 💬 CHAT —
POST  /api/chat/send    [JWT] { message, sessionId? } → { reply, sessionId }
GET    /api/chat/sessions [JWT]                       → { sessions[] }
GET    /api/chat/:id/messages [JWT]                   → { messages[] }

```

Total: **9 endpoint**. Cukup untuk MVP.

⚡ Backend Code (Pseudo / Skeleton)

Gemini Service (Shared)

```

// src/gemini/gemini.service.ts
@Injectable()
export class GeminiService {
  private model: GenerativeModel;

  constructor() {
    const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
    this.model = genAI.getGenerativeModel({ model: 'gemini-2.0-flash' });
  }

  async analyzeImage(imageBase64: string, prompt: string): Promise<string> {
    const result = await this.model.generateContent([
      { text: prompt },
      { inlineData: { mimeType: 'image/jpeg', data: imageBase64 } },
    ]);
    return result.response.text();
  }

  async chat(message: string, history: Content[]): Promise<string> {
    const chat = this.model.startChat({
      history,
      systemInstruction: 'Kamu adalah FarmerBot, asisten pertanian ramah...',
    });
    const result = await chat.sendMessage(message);
    return result.response.text();
  }
}

```

```
}  
}
```

Scan Controller (Ringkas)

```
// src/scan/scan.controller.ts  
@Controller('api/scan')  
@UseGuards(JwtAuthGuard)  
export class ScanController {  
  constructor(private scanService: ScanService) {}  
  
  @Post('analyze')  
  @UseInterceptors(FileInterceptor('image', { dest: './uploads' }))  
  async analyze(  
    @UploadedFile() file: Express.Multer.File,  
    @CurrentUser() user: User,  
  ) {  
    return this.scanService.analyzeAndSave(file, user.id);  
  }  
  
  @Get('history')  
  async history(@CurrentUser() user: User, @Query('limit') limit = 10) {  
    return this.scanService.getUserScans(user.id, +limit);  
  }  
  
  @Get('/:id')  
  async detail(@Param('id') id: string, @CurrentUser() user: User) {  
    return this.scanService.getScanById(id, user.id);  
  }  
}
```

Design (Keep It Simple)

Warna:

- └ Primary: #2E7D32 (hijau tua)
- └ Accent: #FF8F00 (amber/emas)
- └ Danger: #F44336 (merah)
- └ BG: #FAFAFA (putih keabu-abuan)
- └ Text: #212121 (hitam)

Font: Google Sans / Roboto (default Flutter)

Prinsip:

- └ Tombol besar (minimal 48dp)
- └ Font minimal 16sp
- └ Maksimal 3 warna utama
- └ Ikon > teks (petani lebih paham visual)
- └ Jangan bikin ribet, bikin simpel

Checklist MVP (Definition of Done)

WAJIB SELESAI:

- └ [] User bisa register + login (JWT)
- └ [] User bisa foto/pilih gambar tanaman
- └ [] Foto dikirim ke backend → Gemini analisis → tampil diagnosis
- └ [] Diagnosis tampil: nama penyakit + keparahan + rekomendasi
- └ [] User bisa lihat riwayat scan
- └ [] User bisa chat dengan FarmerBot
- └ [] FarmerBot jawab pertanyaan pertanian
- └ [] Backend jalan dan terhubung ke PostgreSQL
- └ [] App jalan di Android tanpa crash

└─ [] APK bisa di-install dan di-demo

BONUS (kalau sempat):

- └─ [] Feedback diagnosis (akurat/tidak)
- └─ [] Share hasil scan ke WhatsApp
- └─ [] Dark mode
- └─ [] Animasi loading yang keren
- └─ [] Quick reply buttons di chat
- └─ [] Deploy backend ke Railway / Render

💰 Biaya (Estimasi 30 Hari)

Item	Biaya
NestJS + PostgreSQL (lokal)	Rp 0
Gemini API (free tier: 15 RPM)	Rp 0
Flutter & Dart	Rp 0
VPS untuk deploy (opsional, Railway free)	Rp 0 - 70.000/bln
Domain (opsional)	~Rp 100.000/tahun
Google Play Console (opsional)	~Rp 400.000 (sekali bayar)
TOTAL	Rp 0 - 570.000

Saat development, backend jalan di localhost . Deploy cuma perlu pas demo/presentasi.

🎓 Yang Bisa Dikembangkan Setelah 30 Hari

Bulan 2: Tambah fitur

- └─ Weather integration (BMKG API)
- └─ Smart Urgency System (pakai Gemini reasoning)
- └─ WhatsApp Bot channel (Twilio)
- └─ Improve prompt accuracy dari data feedback user

Bulan 3: Scale up

- └─ Docker + Docker Compose (backend + DB)
- └─ Deploy ke VPS / cloud proper
- └─ Knowledge base penyakit (RAG + Gemini Embeddings)
- └─ Redis caching untuk response AI
- └─ Publish ke Play Store

vs Kenapa v2 (NestJS) Lebih Baik dari v1 (Firebase)?

Aspek	v1 (Firebase)	v2 (NestJS)
Kontrol	Terbatas oleh Firebase rules	Full kontrol, mau ngapain aja bisa
Database	NoSQL (Firestore) — sulit relasi	PostgreSQL — relasi proper, query powerful
Belajar	Kurang transferable	Skill NestJS berguna di dunia kerja
Cost jangka panjang	Mahal kalau scale (pay per read/write)	Predictable (VPS flat rate)
Auth	Firebase Auth (magic)	JWT manual (paham fundamentalnya)
Deployment	Auto (Firebase hosting)	Perlu setup sendiri (tapi belajar lebih banyak)
Complexity	Lebih gampang di awal	Sedikit lebih ribet, tapi worth it

Ingat: MVP bukan tentang fitur yang banyak.

MVP adalah tentang **1 masalah yang benar-benar terpecahkan**.

Masalah kita: **"Tanaman saya sakit, harus ngapain?"**

Kalau app kita bisa jawab itu dengan scan + AI → sudah cukup.

Sekarang kita punya backend sendiri — siap untuk grow. 🌱